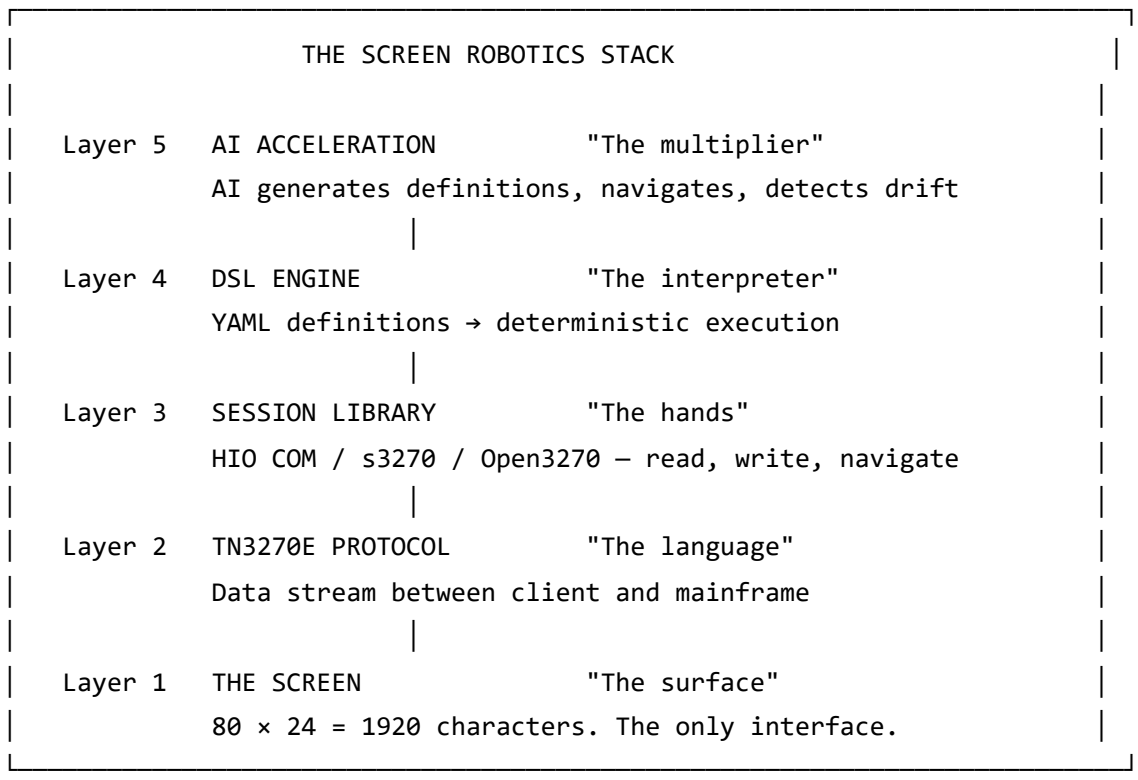
Pattern 4: Screen Robotics — The Complete Reference

> *The door everyone avoided is the one AI walks through most naturally.*
> *Unfiltered. Layered. Print-friendly. For when you need it at hand.*
> *Date: March 8, 2026*

THE BIG PICTURE



Why this pattern? Because it needs ZERO cooperation from whoever owns the mainframe. No source code. No copybooks. No middleware. Just a terminal connection. When every other door is locked — *this one is always open.*

LAYER 1: THE SCREEN — "The Surface"

The Mental Model

Forget everything you know about UIs. A 3270 screen is not a web page. It's a **fixed grid of characters with attributed fields.**

```
Col:  1    10   20   30   40   50   60   70   80
     ┌──────────────────────────────────────────────────────────────────────────┐
R01  │ INQPOL02          POLICY DETAIL INQUIRY                    03/08/2026  │
R02  │                                                                          │
R03  │  POLICY NUMBER:   1234567890                                             │
R04  │  STATUS:          ACTIVE                                                 │
R05  │  INSURED:         JOHN DOE                                               │
R06  │  EFFECTIVE DATE:  01/15/1997                                             │
R07  │  FACE AMOUNT:     $250,000.00                                            │
R08  │  PREMIUM:         $1,245.00                                              │
...  │                                                                          │
R22  │                                                                          │
R23  │  ENTER=NEXT  PF3=EXIT  PF8=MORE                                          │
R24  │                                                                          │
     └──────────────────────────────────────────────────────────────────────────┘

Total: 80 × 24 = 1,920 positions (Model 2)
Each position = one character
Position numbering: 0-based (position 0 = Row 1, Col 1)
```

Offset Arithmetic (Memorize This)

```
offset = (row - 1) × 80 + (col - 1)

Examples:
  Row 1,  Col 1  = offset 0
  Row 1,  Col 80 = offset 79
  Row 3,  Col 20 = offset 179     ← "POLICY NUMBER" value starts here
  Row 24, Col 80 = offset 1919    ← last position on screen

Reverse:
  row = (offset ÷ 80) + 1
  col = (offset % 80) + 1
```

> *Every field on a mainframe screen has a fixed address. This is both the weakness (fragile to layout changes) and the strength (perfectly deterministic, zero ambiguity).*

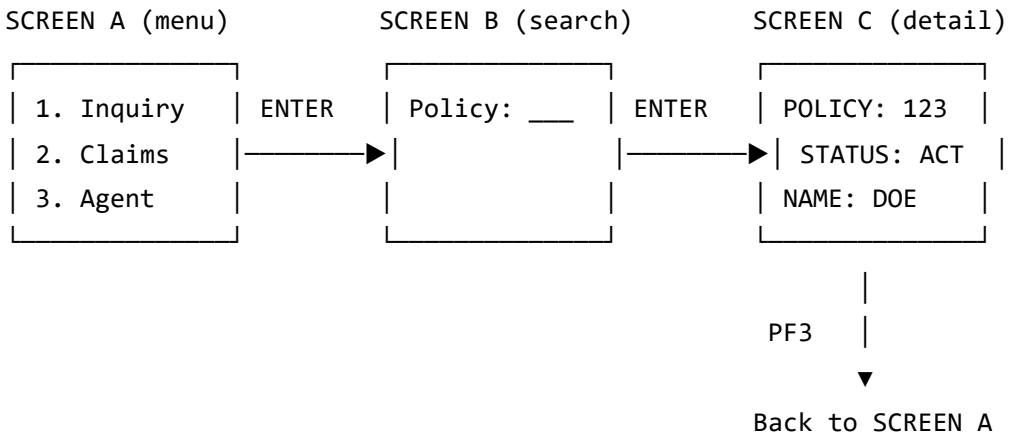
Field Types

Every position on the screen belongs to a **field.** Fields have attributes:

Attribute	Meaning	Your Action
Protected	Read-only (labels, output data)	Read, never write
Unprotected	Input area (user fills this in)	Write here
Hidden	Invisible (passwords)	Write here, can't read
Numeric	Accepts digits only	Validate before sending
High Intensity	Displayed brighter	Often error messages
Modified	User has changed this field	Useful for detecting state

There is **no DOM, no CSS, no event handlers, no JavaScript.** Just characters on a grid with attributes. The screen buffer IS the UI.

The Screen as a State Machine

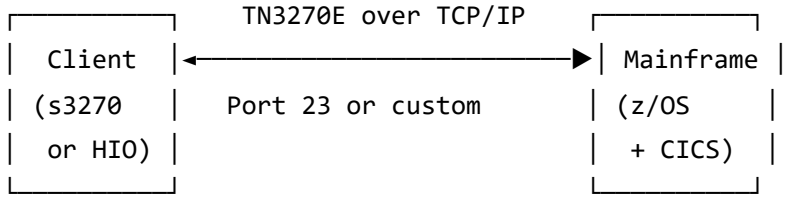


Every screen transition is: **submit AID key → mainframe processes → sends new screen.** The entire application is a state machine navigated by keystrokes.

LAYER 2: TN3270E PROTOCOL — "The Language"

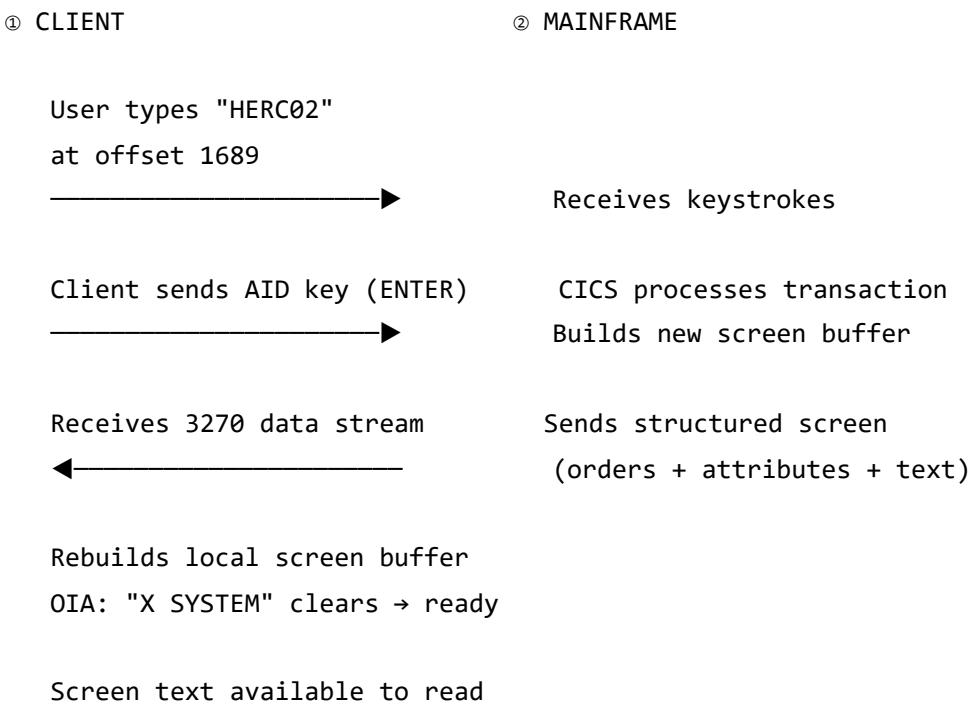
What Is TN3270E

The **wire protocol** between a terminal (real or virtual) and the mainframe. Think of it as HTTP for green screens.



Concept	Detail
Protocol	TN3270E (enhanced) over TCP, defined in RFC 2355
Encoding	EBCDIC on the wire — client library translates to ASCII/Unicode
Data stream	3270DS — structured stream of orders, attributes, text
Session	One TCP connection = one LU (Logical Unit) = one terminal
Model	Model 2 = 80×24. Model 5 = 132×27. Model 2 is standard.
AID keys	Enter, PF1-PF24, PA1-PA3, Clear, Attn — the ONLY ways to "submit"
OIA	Operator Information Area — status bar showing "ready" vs "X SYSTEM"

The Data Flow (One Interaction)



The OIA — Your Traffic Light

OIA states (Operator Information Area — row 25, invisible to screen):

X SYSTEM	Mainframe is processing. DO NOT SEND.
X (other)	Input inhibited for other reason.
(blank/ready)	Safe to type and send.

CRITICAL: After every SendAid, you MUST wait for OIA to clear.
This is WaitForNoX() in HIO / Wait(InputField) in s3270.
Sending keystrokes while X is displayed → data loss or hang.

> *The OIA is the heartbeat of the 3270 conversation. Ignore it and you're typing into the void.*

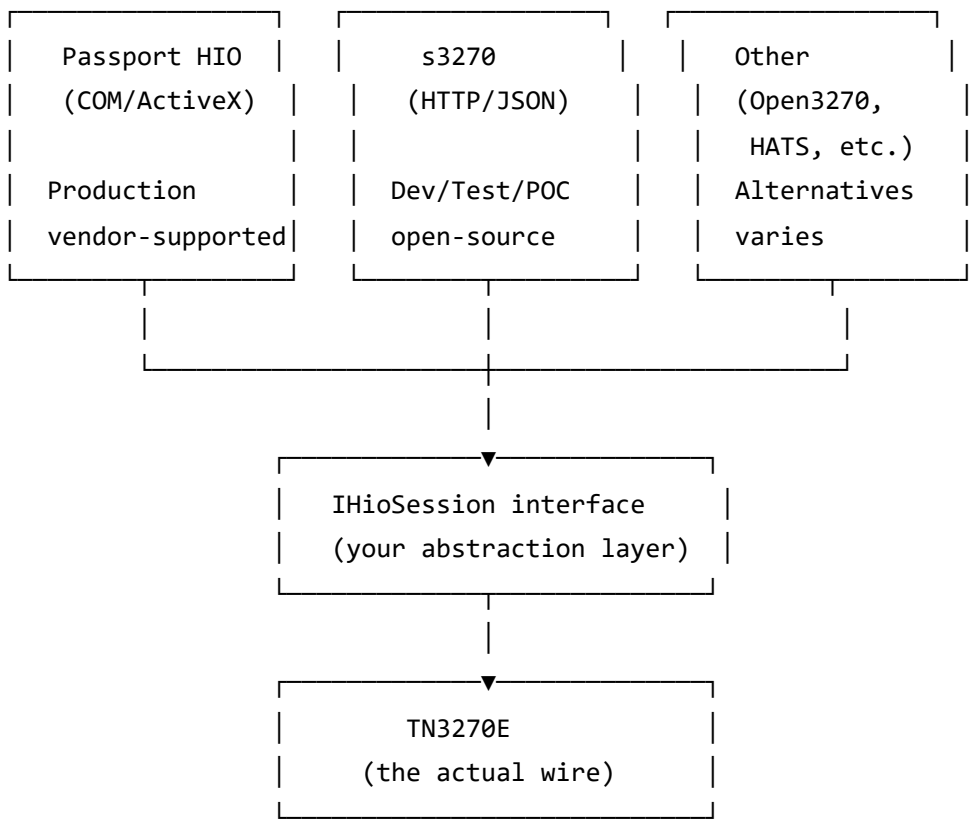
TN3270 vs TN3270E

Feature	TN3270	TN3270E
LU selection	Random	Specific LU name possible

Feature	TN3270	TN3270E
Device type negotiation	Basic	Full (BIND/UNBIND)
Error reporting	None	Structured
SNA support	No	Yes
Use which?	Legacy	**Always TN3270E when available**

LAYER 3: SESSION LIBRARIES — "The Hands"

The Three Contenders



3A: Passport HIO (Production Target)

What: Rocket Software's Host Integration Objects. COM/ActiveX library for .NET. The vendor-supported, production-grade option.

Object hierarchy:

```
Sessions                          ← Collection (static snapshot)
  └─ Session                      ← One host connection (one LU)
       ├─ Screen                  ← The 80×24 virtual screen
       │    ├─ Fields             ← Collection of Field objects
       │    │    └─ Field         ← One field (start, end, attributes, text)
       │    └─ OIA                ← Status (InputInhibited, cursor position)
       └─ Events
            ├─ OnSessionChanged   ← Connected/disconnected
            ├─ OnScreenChanged    ← New screen arrived from host
            └─ OnOIAChanged       ← Ready/busy state changed
```

The lifecycle:

```
① AddSession(zcc_path)     Create from .ZCC config file
② Connect()                Initiate TN3270E connection (non-blocking!)
③ Poll isConnected()       Wait until connection established
④ Screen.WaitForNoX()      Wait for first screen
⑤ ... work ...             Navigate, read, write (see patterns below)
⑥ Disconnect()             End TN3270E session
⑦ CloseSession()           Remove from collection
⑧ Marshal.ReleaseComObject() ← COM cleanup (CRITICAL — memory leaks otherwise)
```

Key methods (the ones that matter 90% of the time):

Method	What It Does	When You Use It
`Screen.String`	Entire screen as one 1920-char string	Screen identification, full-text search
`Screen.FindString(text, start, len)`	Find text position on screen	Locate dynamic content
`Screen.GetData(start, end, PLANE)`	Read character range from screen	Extract field values

Method	What It Does	When You Use It
`Screen.PutString(text, offset)`	Write text at offset	Fill input fields
`Screen.SendKeys(text, offset)`	Simulate typing at offset	Alternative to PutString
`Screen.SendAid(KEY)`	Send Enter/PF/PA key	Submit, navigate, page
`Screen.WaitForStr(text, offset, ms)`	Block until text appears	Screen transition sync
`Screen.WaitForNoX(ms)`	Block until OIA clears	After every SendAid
`Fields.Refresh()`	Reload field collection	After every screen change
`Field.Protected`	Is field read-only?	Distinguish labels vs inputs
`Screen.Cursor`	Current cursor offset	Know where to type

The .ZCC config file:

```
[Connection]
IPHostName=10.1.3.100        # Mainframe IP
TCPPort=23                    # TN3270 port
EmulType=TN3270               # Protocol
TN3270ESupport=Yes            # Use enhanced protocol
ScreenSize=2                  # Model 2 (80×24)
HostCodePage=037              # US EBCDIC
TimeOut=10                    # Connection timeout (sec)
AutoReconnect=Yes             # Reconnect on drop
ExtendedAttributes=Yes        # Color, highlighting
```

Honest assessment (updated March 8, 2026 — after code evaluation):

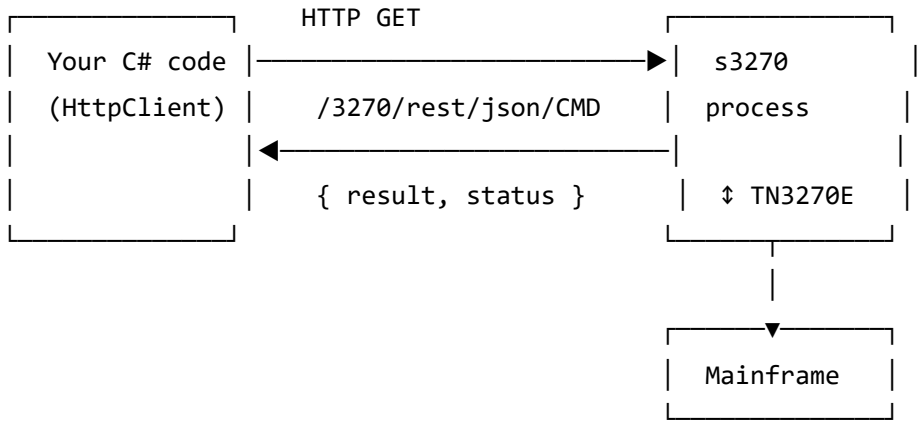
Strength	Weakness
Vendor-supported, battle-tested	COM interop in .NET = painful lifecycle management
Complete API surface	**Currently a stub** — zero working code, all methods throw `NotImplementedException`

Strength	Weakness
Event-driven model available	License tied to session count
OHIO standard-based	Rocket is also a competitor on this deal
.ZCC runtime override	Windows-only (COM = no Linux containers)
Rich field introspection (attrs, types)	STA threading model clashes with async .NET patterns

Current status: HioSession.cs is entirely unimplemented. COM references are commented out. This means HIO has been *planned* but never *proven* in this codebase. That's not a disqualifier — but it is a fact that affects the decision timeline.

3B: s3270 (Your POC Transport)

What: Open-source scripting backend from the x3270 suite. Speaks real TN3270E. Controlled via HTTP/JSON API.



How it works:

```
Launch:   s3270 -httpd 127.0.0.1:9001
Connect:  GET /3270/rest/json/Connect(host:3270)
Type:     GET /3270/rest/json/String(HERC02)
Enter:    GET /3270/rest/json/Enter()
Wait:     GET /3270/rest/json/Wait(3,InputField)
Read:     GET /3270/rest/json/Ascii()     ← full screen text
```

JSON response format:

```
{
  "result": ["line1", "line2", ...],
  "result-err": [false, false, ...],
  "status": "L U U N N 4 24 80 0 0 0x0 ..."
}
```

Key s3270 commands (mapped to HIO equivalents):

s3270 Command	HIO Equivalent	Purpose
`Connect(host:port)`	`Session.Connect()`	Open TN3270E connection
`Disconnect()`	`Session.Disconnect()`	Close connection
`String(text)`	`Screen.SendKeys()` / `PutString()`	Type text
`Enter()`	`Screen.SendAid(ENTER)`	Submit
`PF(n)`	`Screen.SendAid(PFn)`	Function key
`Clear()`	`Screen.SendAid(CLEAR)`	Clear screen
`MoveCursor(r,c)`	(set cursor before PutString)	Position cursor
`Ascii()`	`Screen.String`	Full screen text
`ReadBuffer(Ascii)`	`Screen.GetData()` + `Fields`	Structured read
`Wait(sec,InputField)`	`Screen.WaitForNoX()`	Wait for ready

Honest assessment (updated March 8, 2026 — after code evaluation):

Strength	Weakness
Free, open-source, battle-tested (30+ years)	No vendor support — community only

Strength	Weakness
HTTP API = no COM, no STA threading	Separate process to manage (lifecycle, health checks)
Cross-platform (Linux, macOS, Windows, containers)	No built-in event model (polling-based)
350 lines of working, tested code	Field introspection via ReadBuffer(Field) is partial
JSON responses = AI/Copilot-friendly	No commercial SLA
42 tests passing against live mainframe	Less battle-tested in *enterprise production*

Current status: S3270Session.cs is the only working transport. It handles connection, screen read/write, field parsing (including hex attribute decoding), wait/sync, and cursor management. This is not a prototype — it's a functioning production-quality adapter with proven test coverage.

3C: Comparison Matrix — All Options

Capability	Passport HIO	s3270	IBM HATS	Open3270 (.NET)	Micro Focus
TN3270E protocol	✅	✅	✅	✅	✅
Screen read/write	✅	✅	✅	✅	✅
Field introspection	✅ Full	⚠️ Partial	✅ Full	✅ Basic	✅ Full
Event-driven model	✅	❌	✅	❌	✅
Blocking wait/sync	✅	✅	✅	⚠️	✅
.NET integration	COM interop	HTTP/JSON	Java (WebSphere)	Native .NET	COM/API

Capability	Passport HIO	s3270	IBM HATS	Open3270 (.NET)	Micro Focus
Cross-platform	❌ Windows	✅	❌	✅	❌ Windows
Open source	❌	✅	❌	✅	❌
Vendor support	✅ Rocket	❌ Community	✅ IBM	❌ Community	✅ Micro Focus
License cost	$$$ per session	Free	$$$$	Free	$$$
Production proven	✅ Widely	✅ In tooling	✅ Widely	⚠️ Limited	✅ Widely

The honest take (revised after code evaluation, March 8, 2026):

> *The original plan said HIO for production, s3270 for dev. But today, s3270 has 350 lines of working code and 42 passing tests. HIO has zero. Both speak identical TN3270E protocol — neither has a wire advantage.*
>
> *The smart path: continue with s3270 through POC and early production. Keep IHioSession as the abstraction layer. If a customer or compliance requirement demands vendor-supported tooling, the HIO adapter can be built then — the interface is already defined.*
>
> *Don't add complexity you haven't earned the need for.*

LAYER 4: SCREEN INTERACTION PATTERNS — "The Playbook"

Pattern A: Screen Recognition

The fundamental problem: After pressing a key, *which screen did you land on?*

```
Three recognition strategies:

① MARKER TEXT (most common)
   Look for a known string at a known position.
   "If offset 0-8 contains 'INQPOL02', you're on the Policy Detail screen."

② CONTENT SCAN (flexible)
   Search the entire 1920-char buffer for a unique phrase.
   "If screen contains 'POLICY DETAIL INQUIRY', you're there."

③ FIELD PATTERN (sophisticated)
   Check the arrangement of protected/unprotected fields.
   "If field 3 is unprotected and field 5 is protected, it's the search screen."
```

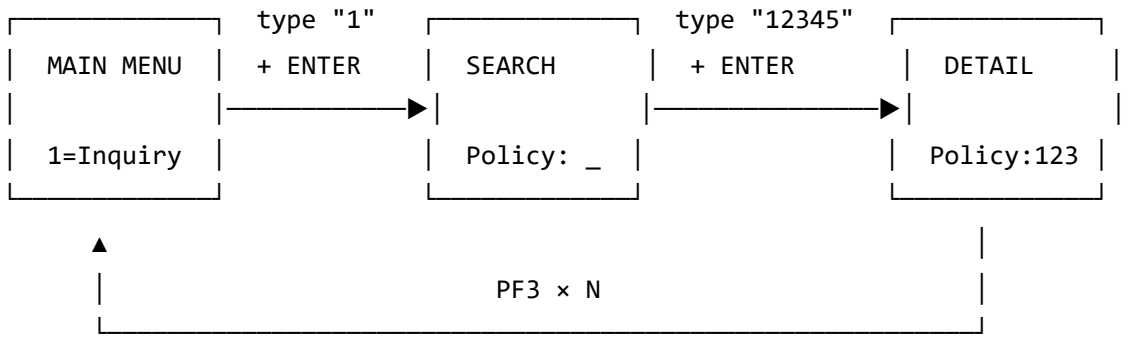
In your YAML DSL:

```yaml
recognition:
  marker_text: "INQPOL02"    # ← Strategy ①
  marker_row: 0               # Check row 0 (top of screen)
```

> *Screen recognition is the immune system of screen robotics. Get it wrong and every subsequent step operates on the wrong data.*

Pattern B: Navigation

Moving between screens = pressing the right keys in the right order.



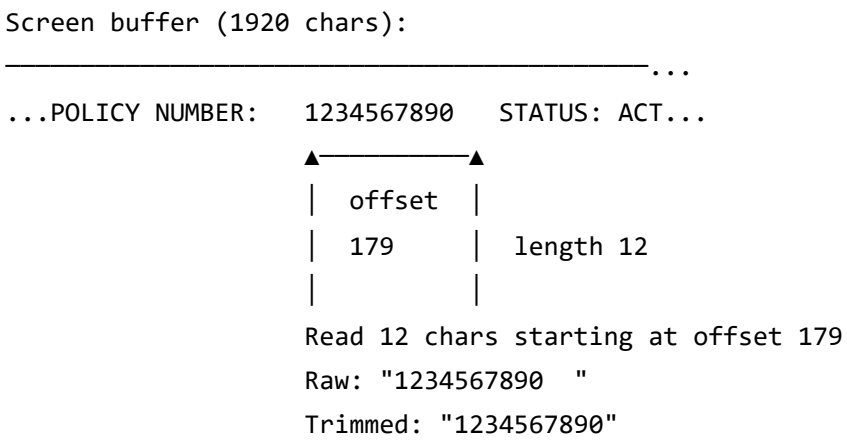
The navigation primitives (your entire vocabulary):

Action	Method	When
Type text at position	`PutString(text, offset)`	Fill a field
Type text at cursor	`SendKeys(text, cursor)`	When field offset unknown
Submit / Confirm	`SendAid(ENTER)`	Process input
Go back / Exit	`SendAid(PF3)`	Return to previous screen
Page down (more data)	`SendAid(PF8)`	Tables, scroll forward
Page up	`SendAid(PF7)`	Scroll backward
Clear screen	`SendAid(CLEAR)`	Reset terminal state
Tab to next field	`SendAid(TAB)`	Move between input fields
Wait for ready	`WaitForNoX(ms)`	**AFTER EVERY SendAid**
Wait for specific text	`WaitForStr(text, offset, ms)`	Confirm screen arrived

> *Navigation is a conversation with the mainframe. You speak in keystrokes. It responds in screens. Miss a beat (skip a WaitForNoX) and the conversation collapses.*

Pattern C: Data Extraction — Single Fields

Reading one field from the screen:



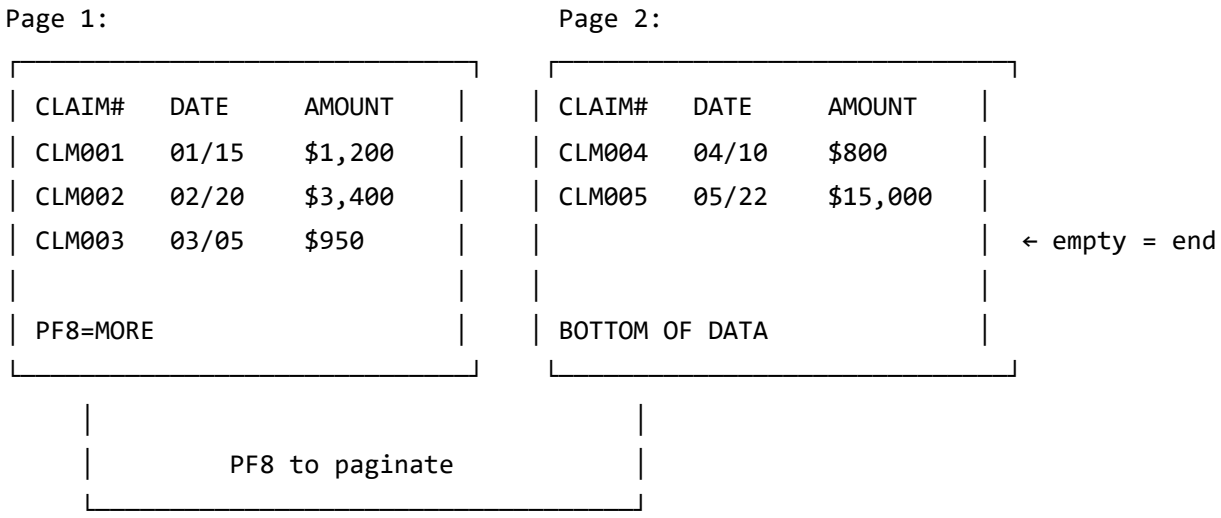
Type conversion matters:

Screen Text	Field Type	Converted Value
`"1234567890  "`	string	`"1234567890"`
`"$1,245.00   "`	currency	`1245.00` (decimal)
`"01/15/1997   "`	date (MM/dd/yyyy)	`"1997-01-15"`
`"ACTIVE      "`	string	`"ACTIVE"`
`"            "`	nullable string	`null`
`"Y           "`	boolean	`true`

> *The mainframe right-pads everything with spaces. Every field read needs trimming. Every type needs conversion. This is the tax you pay for Pattern 4.*

Pattern D: Data Extraction — Tables (The Hard One)

Tabular data spans rows AND pages. This is the most complex pattern.



The extraction loop:

```
1. Read rows from defined region (start_row → end_row)
2. For each row: read each column at its offset + length
3. Detect empty rows (blank? check_column empty? marker?)
4. Check: is this the last page?
   - "BOTTOM OF DATA" text visible?
   - Same screen as before PF8? (no new data)
   - Hit max pages limit?
5. If more pages: SendAid(PF8) → WaitForNoX → goto step 1
6. Merge all pages into single list
```

End-of-data detection strategies:

Strategy	How	Reliability
Marker text	Look for "BOTTOM OF DATA" or "END"	High — if consistent
Same screen	Compare screen text before/after PF8	High — universal fallback
Empty rows	First empty row = end of data on page	Medium — blank rows ≠ always end
Max pages	Safety cap (e.g., 50 pages)	Safety net — always set this

> *Table extraction is where screen robotics earns its "dirty" reputation. But it's a solved pattern — once you have the loop, it works for every table on every screen.*

Pattern E: Branching (On-Screen Decisions)

After submitting, multiple screens are possible:

```
Enter policy number
      │
      ├──▶ Screen contains "INQPOL02"    → SUCCESS: read detail
      │
      ├──▶ Screen contains "NOT FOUND"   → ERROR: return structured error
      │
      └──▶ Screen contains "SYSTEM BUSY" → RETRY: wait, try again
```

In your YAML DSL:

```yaml
on_screen:
  - when:
      contains_text: "INQPOL02"
    goto: "read-policy-detail"
  - when:
      contains_text: "POLICY NOT FOUND"
    goto: "handle-not-found"
```

> *Branching is what separates a "screen scraper" from a "screen robot." A scraper reads. A robot makes decisions.*

Pattern F: Error Detection

Mainframe errors appear as text on the screen, usually in a consistent zone:

```
Row 20-23: The "error zone" on most CICS screens

┌──────────────────────────────────────────────────┐
│  ...                                             │
│  Row 20:                                         │
│  Row 21: *** POLICY NOT FOUND ***                │  ← Error text here
│  Row 22:                                         │
│  Row 23: PF3=EXIT                                │
└──────────────────────────────────────────────────┘
```

Your YAML captures this:

```yaml
error_indicators:
  - text: "POLICY NOT FOUND"
    row_range: [20, 23]
    action: "return_error"
  - text: "INVALID POLICY"
    row_range: [20, 23]
    action: "return_error"
```

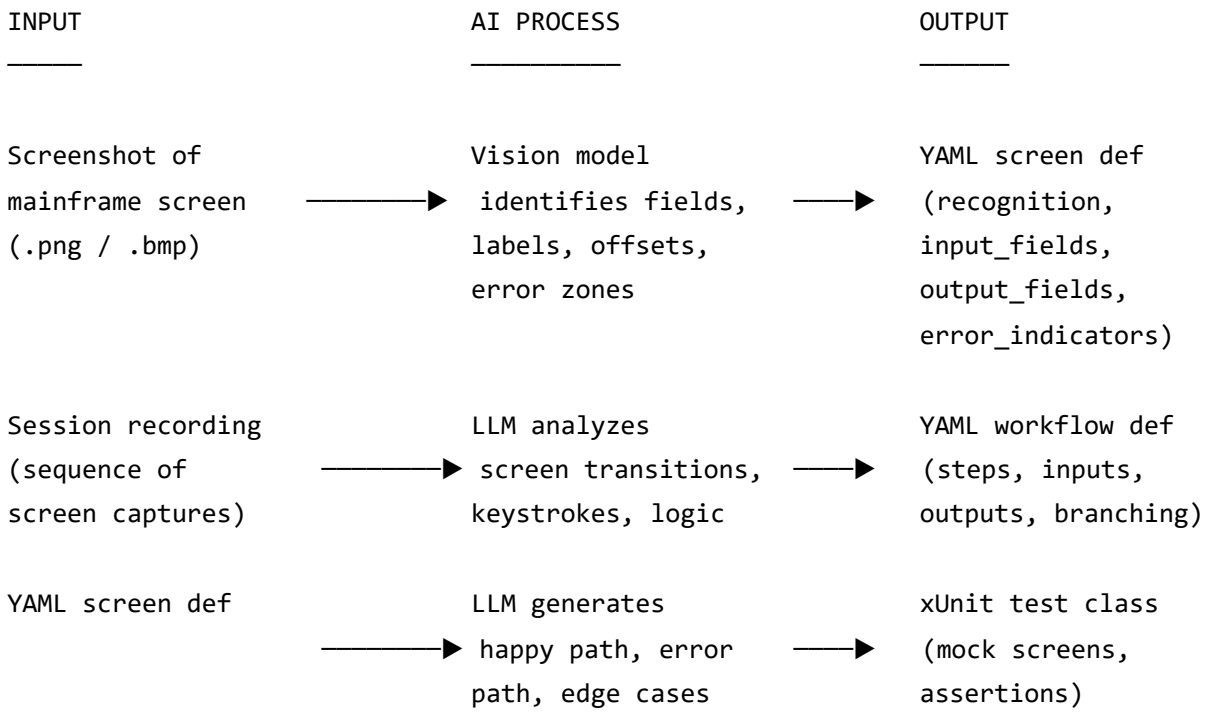
> *Errors on mainframes are text, not HTTP status codes. Your screen robot must read the screen for errors the same way a human operator does — by scanning the bottom rows.*

LAYER 5: AI ACCELERATION — "The Multiplier"

Where AI Plugs Into Each Layer

Layer 1 (Screen)	AI VISION reads screen captures → extracts field layout "Show me a screenshot, I'll tell you every field's position"
Layer 2 (Protocol)	AI doesn't touch this. Protocol is protocol.
Layer 3 (Library)	AI GENERATES adapter code (Copilot writes S3270Session.cs)
Layer 4 (Engine)	AI GENERATES YAML definitions from screen captures AI GENERATES test cases from definitions AI GENERATES workflow sequences from session recordings
Layer 5 (Runtime)	AI AGENT navigates screens autonomously (future state) AI MONITORS for screen drift (compare reality vs YAML)

The AI Generation Pipeline



The Three AI Operating Modes

Mode	When	Determinism	Compliance-Safe
AI generates, human validates	Dev-time definition creation	✅ Output is YAML (deterministic)	✅ Yes
AI monitors, human acts	Production drift detection	✅ Alert only, no auto-change	✅ Yes
AI navigates autonomously	Future: agentic screen robot	❌ Non-deterministic	⚠️ Needs guardrails

> *The current sweet spot: AI at dev-time (generate 80% of definitions), deterministic engine at runtime (100% auditable). The agentic future is coming but insurance compliance isn't ready for it today.*

AI + Screen Robotics: Why This Pairing Is Unique

```
WHY OTHER PATTERNS DON'T BENEFIT AS MUCH FROM AI:

Pattern 1 (Direct API):   AI helps write code, but the hard part is
                           getting access — AI can't unlock a door.

Pattern 2 (Messaging):    AI helps with message formats, but the
                           bottleneck is mainframe-side queue handlers.

Pattern 3 (File/CDC):     AI helps with data mapping, but the hard
                           part is COMP-3 decoding and EBCDIC handling.

Pattern 4 (Screen Robot): The ENTIRE workflow is screen-readable.
                           AI can see exactly what a human sees.
                           AI can generate definitions from screenshots.
                           AI can navigate like a human — but faster.

∴ Screen robotics is the most AI-native integration pattern.
  The "dirtiest" pattern is paradoxically the most AI-friendly.
```

YOUR POC: WHERE IT MAPS

The Architecture Mapping

```
THIS REFERENCE                YOUR POC
──────────────                ────────
Layer 1 (Screen)          →   YAML screen definitions (insurance-tests/screens/)
Layer 2 (TN3270E)         →   TK5/Hercules MVS 3.8j on localhost:3270
Layer 3 (s3270)           →   S3270Session.cs — HTTP/JSON to s3270 process (WORKING — 350 lir
Layer 3 (HIO)             →   HioSession.cs — COM stub (unimplemented — available if needed)
Layer 3 (abstraction)     →   IHioSession.cs — transport-agnostic interface (swap-ready)
Layer 4 (engine)          →   WorkflowExecutor → StepExecutor → ScreenReader → TableExtractor
Layer 4 (definitions)     →   YAML screen defs + workflow defs (data, not code)
Layer 5 (AI)              →   Copilot-generated engine code + definitions
```

What's Proven

Layer	Evidence	Tests
Screen reading	1920-char buffer read correctly from live TK5	ScreenReaderTests (8)
Table extraction	Pagination, empty rows, currency parsing	TableExtractorTests (6)
Navigation	Enter, PF3, PF8, Clear — screen transitions verified	S3270IntegrationTests (6)
Multi-screen flow	TSO login → ISPF → exit (6 screens)	TsoLoginWorkflowTests (2)
Branching	on_screen condition routing	WorkflowExecutorTests (5)
Error detection	Screen text scanning in defined row ranges	StepExecutorTests (12)
Total	**42 tests passing against live mainframe**	

QUICK REFERENCE CARDS (tear-off)

Card 1: The Interaction Cycle

```
┌──────────────────────────────────────────┐
│  THE CYCLE (every single screen interaction) │
│                                          │
│  1. RECOGNIZE  — Which screen am I on?   │
│  2. CHECK      — Any errors?             │
│  3. READ       — Extract output fields   │
│  4. WRITE      — Fill input fields        │
│  5. SUBMIT     — SendAid (Enter/PF key)  │
│  6. WAIT       — WaitForNoX (MANDATORY)  │
│  7. GOTO 1                               │
└──────────────────────────────────────────┘
```

Card 2: Offset Cheat Sheet (Model 2: 80×24)

```
offset = (row - 1) × 80 + (col - 1)

Row  1, Col  1 = 0        Row  1, Col 80 = 79
Row  2, Col  1 = 80       Row 12, Col 40 = 919
Row 22, Col 11 = 1690     Row 24, Col 80 = 1919

Reverse: row = (offset ÷ 80) + 1,  col = (offset % 80) + 1
```

Card 3: Key Mapping

```
ENTER  = Submit / Confirm       PF1  = Help
PF3    = Exit / Back             PF7  = Page Up
PF8    = Page Down               PF12 = Cancel
CLEAR  = Reset terminal          TAB  = Next field
ATTN   = Attention (interrupt)   PA1/PA2 = Program Attention
```

Card 4: The Honest Truth About Pattern 4

```
┌─ STRENGTHS ──────────────────────────────────┐
│ ✅ Needs ZERO mainframe cooperation          │
│ ✅ Works with any mainframe, any app         │
│ ✅ The most AI-native pattern                │
│ ✅ Your POC: 42 tests proving it works       │
├─ WEAKNESSES ─────────────────────────────────┤
│ ⚠️  Fragile to screen layout changes         │
│ ⚠️  Slower than direct API (screen I/O)      │
│ ⚠️  Offset math = precision required         │
│ ⚠️  No type safety (everything is text)      │
├─ THE MITIGATION ─────────────────────────────┤
│ YAML definitions absorb the fragility.       │
│ Screen changes = update YAML, not code.      │
│ AI detects drift before users do.            │
│ The engine is deterministic and auditable.   │
└──────────────────────────────────────────────┘
```

Card 5: The Screen Robot vs Screen Scraper

SCRAPER	ROBOT
Reads data from screens	Reads, writes, navigates, decides
One-way extraction	Full round-trip interaction
Breaks on any change	YAML absorbs changes
No error handling	Detects errors, branches, retries
Script per screen	One engine, data-driven

Your engine is a ROBOT, not a scraper.

Card 6: When To Use Which Library (revised March 8)

CONTEXT	USE
Unit tests	MockHioSession (no network)
Dev / POC	s3270 (free, proven, cross-platform)
Production	s3270 (working today) OR HIO (if needed)
Linux/Cloud	s3270 (only option — no COM dependency)
Debugging	x3270 GUI (visual terminal)
Enterprise	HIO if customer requires vendor SLA

All speak through IHioSession. Swap adapter, zero code changes.
Decision trigger: only invest in HIO when a concrete requirement demands it.

Card 7: The YAML Definition Anatomy

```
screen_id: "INQPOL02"           ← Unique name
recognition:
  marker_text: "INQPOL02"       ← How to know you're here
input_fields:
  - name: policy_number          ← What to type
    row: 4, col: 23, length: 12
output_fields:
  - name: status                 ← What to read
    row: 5, col: 20, length: 10
    type: string
error_indicators:
  - text: "NOT FOUND"            ← What to watch for
    row_range: [20, 23]
```

ONE file per screen. ONE engine for all files.
AI generates these. Humans validate.

Card 8: The Five Things That Go Wrong

PROBLEM	SYMPTOM	FIX
1. Missed WaitForNoX	Data loss / hang	ALWAYS wait after Send
2. Wrong screen recognized	Reads garbage fields	Better marker text
3. Offset off by one	Shifted field values	Verify with x3270 GUI
4. Screen layout changed	Fields misaligned	Update YAML, not code
5. Timeout too short	False negatives	Increase wait, add log

Card 9: The Competitive Framing

THEM (IBM / Rocket / Micro Focus)	YOU (AI-First Screen Robot)
52 hand-coded programs	1 engine + 52 YAML files
Months of development	AI generates 80% in days
Screen change = code change	Screen change = YAML update
Needs mainframe specialists	Needs YAML + one .NET dev
No AI differentiation	AI is the architecture
Product license per seat	Custom-built, you own it

"Pattern 4 is the service entrance — unglamorous, through the kitchen. But it's the one that's always unlocked. And with AI holding the flashlight, it turns out the kitchen has everything you need."